Attention 的数学推导

像数学课本一样，从「想做什么」出发，一步一步推到 QKV 与多头

2026 年 4 月 23 日

摘要

本讲义不抢图、不堆术语，而是严格按照数学课本风格走一遍 attention 的推导：先把要解决的目标写清楚，写出建模假设，然后从最朴素的尝试开始，**每改进一步都说明「为什么必须这么改」**，直到自然推出 Transformer 论文里那个完整的多头注意力公式。

读完之后，你应该能在白板上不查资料地写出 $\mathrm{Att}(Q,K,V)=\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ 的每一项是怎么来的、为什么这么来。

目录

1 第 0 步：先把目标写清楚 **3**

2 第一次尝试：直接用相似度加权 **3**

2.1 动机 3

2.2 矩阵化 4

2.3 它满足目标的几条吗？ 4

3 为什么必须引入 W_Q, W_K, W_V **4**

3.1 尝试 1 的两个根本缺陷 4

3.2 解药：把每个 token 投影成三个不同的角色 5

3.3 重新定义打分函数 5

3.4 得到 attention 的雏形 5

4 为什么要除以 $\sqrt{d_k}$ **6**

4.1 问题：维度变大会让 softmax「过激」 6

4.2 尖锐分布的副作用：梯度消失 6

4.3 解药：把分数除以 $\sqrt{d_k}$ 6

4.4 完整的「缩放点积注意力」 6

5 为什么需要「多头」 **6**

5.1 单头的局限 6

5.2 把「想要 h 套独立关注」精确化 7

5.3 方案 A（笨办法）：每个头都开「全宽」 7

5.4 方案 B（聪明办法）：把同一份预算切成 h 段 8
5.5 方案 A vs 方案 B 一图看完 8
5.6 为什么「砍小子空间」不致命? 8
5.7 参数量核算（B 的精确账） 9
5.8 为什么这等价于「reshape + transpose」 9

6 把整套推导收成一个清单 10

1 第 0 步：先把目标写清楚

我们手上有的对象很简单：

定义 1 (输入)**.** 一段长度为 n 的文本，每个 token 已经被嵌入成 d 维向量。把这些向量按行堆起来得到矩阵

$$X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n\times d}.$$

$x_i \in \mathbb{R}^d$ 是第 i 个 token 的输入向量。

目标 1 (本章要做的事)**.** 设计一个函数 $f : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$，把输入 X 变成输出

$$Y = f(X) \in \mathbb{R}^{n\times d},$$

满足：

1. **上下文感知**：y_i（输出的第 i 行）必须能依赖所有 $x_1, \ldots, x_n$，而不只是 x_i；
2. **置换等变**（除位置编码外）：先打乱再算等价于先算再打乱；
3. **可微**：f 关于参数处处可导，能用反向传播训练；
4. **并行**：所有 y_i 必须能在 GPU 上一次性算出来，而不是循环。

备注. **条件 1** 排除了「$y_i = g(x_i)$」这种逐位置网络（FFN）；**条件 4** 排除了 RNN（必须按顺序算）。满足这 4 条的最简单方案，就是 attention。

2 第一次尝试：直接用相似度加权

2.1 动机

最朴素的想法：要让 y_i 看到所有 x_j，就把它们加起来。但「均匀加和」会丢失结构，所以要加权。权重应该反映「x_j 对 x_i 有多重要」。

怎么量化「重要」？在向量空间里最常用的相似度就是点积：两个向量越像，点积越大。

尝试 1 (无参数注意力)**.** 对每对位置 (i, j)，定义未归一化分数

$$e_{ij} = x_i^\top x_j \in \mathbb{R}.$$

对每一行做 softmax 把分数变成「合法权重」：

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})}, \qquad \sum_{j=1}^n \alpha_{ij} = 1.$$

最终输出是加权求和：

$$y_i = \sum_{j=1}^n \alpha_{ij}\, x_j.$$

2.2 矩阵化

把上面写成矩阵形式：

$$E = XX^\top \in \mathbb{R}^{n\times n}, \qquad A = \mathrm{softmax}_{\text{行}}(E), \qquad Y = AX.$$

逐项验证：

- $E_{ij} = x_i^\top x_j$
- $A_{ij} = \alpha_{ij}$ （softmax 沿每一行做）
- $(AX)_{i,:} = \sum_j A_{ij}\, x_j^\top = y_i^\top$

2.3 它满足目标的几条吗？

- 上下文感知 （每个 y_i 都聚合了所有 x_j）
- 置换等变 （公式里没有显式位置）
- 可微 （softmax 与矩阵乘都是可导的）
- 并行 （一次矩阵乘搞定）

看起来全都满足了。那为什么还要 QKV？
下一节回答这个问题。

3 为什么必须引入 W_Q, W_K, W_V

3.1 尝试 1 的两个根本缺陷

问题 1 (对称性导致角色混淆)**.** 注意尝试 1 里的相似度矩阵 $E = XX^\top$ 是对称的：

$$E_{ij} = x_i^\top x_j = x_j^\top x_i = E_{ji}.$$

但语言里「我看你」和「你看我」常常不该一样重要：比如代词 *it* 在英文里要指向某个名词，但那个名词不一定要回看 *it*。对称的相似度无法表达这种非对称的关注。

问题 2 (「相似度」与「贡献」被强行绑死)**.** 在尝试 1 里，被加权的对象 x_j 既参与打分（出现在 $E_{ij} = x_i^\top x_j$），又参与被聚合（出现在 $\sum_j \alpha_{ij} x_j$）。这意味着「x_j 让自己显眼的方式」和「x_j 真正贡献的内容」必须由同一个向量承担。

这是不必要的限制： 有时一个 token 的「让人找到我用什么标签」(key) 和「我能贡献什么内容」(value) 应当是两件事。

问题 3 (模型没有任何可调旋钮)**.** $Y = \mathrm{softmax}(XX^\top)X$ 里没有任何参数。模型完全无法根据任务学到「在这个任务里，什么算重要」。

3.2 解药：把每个 token 投影成三个不同的角色

为了同时解决以上三个问题，最经济的改动是：**给每个 x_i 派三份不同的「替身」**，分别承担三种职能。

定义 2 (Query / Key / Value 投影)**.** 引入三个可学习矩阵

$$W_Q \in \mathbb{R}^{d\times d_k}, \quad W_K \in \mathbb{R}^{d\times d_k}, \quad W_V \in \mathbb{R}^{d\times d_v}.$$

对每个 token 定义

$$q_i \;=\; W_Q^\top x_i, \qquad k_j \;=\; W_K^\top x_j, \qquad v_j \;=\; W_V^\top x_j.$$

矩阵化：$Q = XW_Q$，$K = XW_K$，$V = XW_V$，形状 $Q, K \in \mathbb{R}^{n\times d_k}$，$V \in \mathbb{R}^{n\times d_v}$。

每个矩阵的物理意义（这是理解 QKV 的关键）：

- W_Q：「我以什么方式询问别人」——把 x_i 投到查询空间；
- W_K：「我以什么方式被找到」——把 x_j 投到钥匙空间；
- W_V：「如果真的有人来取，我贡献什么」——把 x_j 投到内容空间。

3.3 重新定义打分函数

把尝试 1 里的 $e_{ij} = x_i^\top x_j$ 换成

$$e_{ij} \;=\; q_i^\top k_j \;=\; (W_Q^\top x_i)^\top (W_K^\top x_j) \;=\; x_i^\top W_Q W_K^\top x_j. \tag{1}$$

引理 1 (非对称性已经获得)**.** 一般情况下 $W_Q W_K^\top \neq W_K W_Q^\top$，所以

$$e_{ij} = x_i^\top (W_Q W_K^\top) x_j \;\neq\; x_j^\top (W_Q W_K^\top) x_i = e_{ji}.$$

即 **打分矩阵** $E = QK^\top$ **不再强制对称**，模型可以学到「我看你 $\neq$ 你看我」的关注模式。

引理 2 (「钥匙」和「内容」已经分离)**.** 打分用的是 $k_j = W_K^\top x_j$，聚合用的是 $v_j = W_V^\top x_j$。两个投影矩阵独立，所以「让别人找到我用什么标签」和「我贡献什么」是两份独立的可学习内容。

3.4 得到 attention 的雏形

把上面三件事写到一起：

$$E = QK^\top \in \mathbb{R}^{n\times n}, \qquad A = \mathrm{softmax}_{\text{行}}(E), \qquad Y = AV \in \mathbb{R}^{n\times d_v}.$$

合成一个公式：

$$\boxed{\mathrm{Att}(Q,K,V) \;=\; \mathrm{softmax}(QK^\top)\,V.} \tag{2}$$

到这一步还差一个细节：分母 $\sqrt{d_k}$。下一节回答它。

4 为什么要除以 $\sqrt{d_k}$

4.1 问题：维度变大会让 softmax「过激」

引理 3 (点积的方差随维度增长)**.** 设 $q, k \in \mathbb{R}^{d_k}$ 的各分量独立同分布，均值 0，方差 1。则

$$\mathbb{E}[q^\top k] \;=\; 0, \qquad \mathrm{Var}(q^\top k) \;=\; \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) \;=\; d_k.$$

也就是说，当 $d_k = 64$ 时，分数的标准差大约是 8；当 $d_k = 512$ 时，标准差大约是 $\sqrt{512} \approx 22.6$。分数越大，softmax 出来的概率分布越尖（接近 one-hot）。

4.2 尖锐分布的副作用：梯度消失

设 softmax 的输入为 $z = (z_1, \ldots, z_n)$，输出为 p，则

$$\frac{\partial p_i}{\partial z_j} \;=\; p_i(\delta_{ij} - p_j).$$

当某个 $p_i \to 1$、其它 $\to 0$ 时，$p_i(1 - p_i) \to 0$，所有偏导都趋于 0。**反向传播时梯度几乎为零，模型学不动**。

4.3 解药：把分数除以 $\sqrt{d_k}$

把分数归一化到与 d_k 无关的尺度：

$$\tilde{e}_{ij} \;=\; \frac{q_i^\top k_j}{\sqrt{d_k}}, \qquad \mathrm{Var}(\tilde{e}_{ij}) \;=\; \frac{d_k}{(\sqrt{d_k})^2} = 1.$$

无论 d_k 是 64 还是 512，分数的尺度都稳定在 $O(1)$，softmax 不会过早饱和。

4.4 完整的「缩放点积注意力」

把缩放装回去：

命题 1 (Scaled Dot-Product Attention)**.**

$$\boxed{\mathrm{Att}(Q, K, V) \;=\; \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.}$$

本节交付

到此为止我们已经把 §3.3 与 §3.4 全部推完了：**每一步都不是天上掉下来的**——QKV 是为了打破对称性、分离 key 与 value，并引入参数；$\sqrt{d_k}$ 是为了 softmax 在不同维度下都能保持温度。

5 为什么需要「多头」

5.1 单头的局限

单头注意力的输出（不算 W_V）维度是 $n \times d_v$。设想我们让 $d_v = d$（保持模型宽度不变）。此时 $W_Q, W_K \in \mathbb{R}^{d \times d_k}$。

关键观察：注意力矩阵 $A = \mathrm{softmax}(QK^\top/\sqrt{d_k})$ 的形状是 $n \times n$，也就是「每个位置看哪里」是一份权重表。**一份权重表只能编码一种关注模式。**

但语言里每个 token 同时存在多种关系：

- 主语 → 谓语；
- 形容词 → 它修饰的名词；
- 代词 → 指代对象；
- 邻居关系 / 句法依存 / 同义对……

我们希望模型能同时学若干份独立的关注模式。

5.2 把「想要 h 套独立关注」精确化

「同时学 h 种关注模式」翻译成数学，就是要 h 张各不相同的注意力矩阵：

$$A^{(1)}, A^{(2)}, \ldots, A^{(h)} \in \mathbb{R}^{n\times n}.$$

每张 $A^{(i)}$ 对应一种「看哪里」的规律。对应地，我们也要 h 段独立的输出 $\mathrm{out}^{(i)} = A^{(i)}V^{(i)}$，最后还得有办法把这 h 段拼成一段完整的输出。

备注. **两个方案的共同点：**都给每个头一套独立的 (W_Q^i, W_K^i, W_V^i)，都得到 h 张独立的 $A^{(i)}$。
两个方案的唯一区别：每个头用多大的 d_k 和 d_v——也就是「每个头看的子空间有多宽」。

5.3 方案 A（笨办法）：每个头都开「全宽」

最直接的做法：让每个头都和原来的单头一样宽，即

$$d_k^{(A)} = d_v^{(A)} = d_{\mathrm{model}}.$$

形状：

$$W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{\mathrm{model}}\times d_{\mathrm{model}}}, \qquad i = 1, \ldots, h.$$

每个头：

$$\mathrm{head}_i^{(A)} = \mathrm{Att}(XW_Q^i,\, XW_K^i,\, XW_V^i) \in \mathbb{R}^{n\times d_{\mathrm{model}}}.$$

代价核算（投影矩阵的总参数量）：

$$\underbrace{3d_{\mathrm{model}}^2}_{\text{单头}} \to \underbrace{h\cdot 3d_{\mathrm{model}}^2}_{h\text{ 个全宽头}}.$$

当 $h = 8, d_{\mathrm{model}} = 512$ 时，光是 QKV 投影就从 $\approx 0.79\,\mathrm{M}$ 涨到 $\approx 6.3\,\mathrm{M}$，*每层都要乘以 8*。计算量同样涨 8 倍。**贵到不能接受。**

小结. 方案 A：「我想要 h 个头」就字面上叠 h 个。做了同样的事 h 次，参数和算量都变 h 倍。

5.4 方案 B（聪明办法）：把同一份预算切成 h 段

关键观察：方案 A 之所以贵，是因为它不肯把预算切开。如果我们承认「单头能用的参数预算就是 $3d_{\text{model}}^2$」，那么最自然的折中是：

> 把这 $3d_{model}^2$ 的预算**切成 h 份**，每个头分到 $1/h$。每个头看到的子空间维度 d_k 也按比例缩小到 d_{model}/h。

定义 3 (多头注意力)**.** 设 $d_k = d_v = d_{\text{model}}/h$。每个头的形状变成：

$$W_Q^i, W_K^i \in \mathbb{R}^{d_{\text{model}} \times d_k}, \qquad W_V^i \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

每个头：

$$\text{head}_i = \text{Att}\big(XW_Q^i,\ XW_K^i,\ XW_V^i\big) \in \mathbb{R}^{n \times d_v}, \qquad i = 1, \ldots, h.$$

然后拼接 + 一次输出投影：

$$\boxed{\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.}$$

拼接后的形状是 $n \times (h \cdot d_v) = n \times d_{\text{model}}$，$W_O$ 把它映回 d_{model} 维。

5.5 方案 A vs 方案 B 一图看完

	方案 A（全宽）	方案 B（切片，Transformer 用的）
$d_k = d_v$	d_{model}	d_{model}/h
单个 W_Q^i 的形状	$d_{\text{model}} \times d_{\text{model}}$	$d_{\text{model}} \times (d_{\text{model}}/h)$
h 套 QKV 总参数量	$h \cdot 3d_{\text{model}}^2$	$3d_{\text{model}}^2$
h 张注意力矩阵 $A^{(i)}$	有	有
每张 $A^{(i)}$ 表达力	在完整 d_{model} 维比对	只在 $d_k = d_{\text{model}}/h$ 维子空间比对
h 头是否独立学不同关注	是	是
拼接后维度	$h \cdot d_{\text{model}}$	d_{model}
是否需要 W_O 融合	需要（且必须把 $h \cdot d_{\text{model}}$ 压回去）	需要（$d_{\text{model}} \to d_{\text{model}}$）
和单头比，参数额外开销	$\approx 3(h-1)d_{\text{model}}^2 + hd_{\text{model}}^2$	只多了 W_O，约 d_{model}^2

关键结论：方案 B 用「把每个头看的子空间变窄」这一个让步，换来「总参数和总算量与单头几乎一样」。**我们没有放弃「h 张独立注意力」这一核心目标**——A 和 B 都拿到了 h 张独立的 $A^{(i)}$，区别只是每张 $A^{(i)}$ 是在多大的子空间里算出来的。

5.6 为什么「砍小子空间」不致命？

读到这里你应该会有一个自然怀疑：

把 d_k 从 512 砍到 64，每个头看到的信息变少了 8 倍，难道整体表达力不也跟着崩了 8 倍？

不会。三条理由，按重要性排：

1. **我们要的本来就不是「每个头看见所有维度」，而是「每个头专注一种关系」**。一种语法关系（比如「主语—谓语」）通常只需要少数几个维度就能编码，不需要每个头都把整段 d_{model} 重新审一遍。让头变窄，反而逼它专精。

2. **h 份子空间合起来是完整的 d_{model}**。因为 $h \cdot d_v = d_{\text{model}}$，$h$ 个头各看一段不重合的子空间，合起来正好覆盖整个 d_{model} 维。没有信息总量上的损失，只是分工方式不同。

3. **W_O 把分工的产出再融合回去**。每个头的输出在拼接之后只是并排放在一起，W_O 让模型可以学到「在最终输出里按什么权重混这 h 个头」，恢复跨子空间的信息互动。

小结. 方案 A 是「用蛮力的钱解决问题」；方案 B 是「承认预算固定，用分工解决问题」。两者目标完全一致（拿到 h 张独立的注意力），只是 B 在每个头的「视野宽度」上做了让步，换回了「便宜得起」。Transformer 选 B。

5.7 参数量核算（B 的精确账）

引理 4 (B 的总参数量与单头几乎相同)**.**

$$\sum_{i=1}^{h} \dim(W_Q^i) = h \cdot d_{\text{model}} \cdot \frac{d_{\text{model}}}{h} = d_{\text{model}}^2.$$

W_K, W_V 同理。所以「单头用整个 d_{model}」与「h 头各用 $d_k = d_{\text{model}}/h$」主体投影的总参数量完全相同，都是 $3d_{\text{model}}^2$。唯一额外的开销是输出投影 W_O，多出 d_{model}^2。

直觉. 所以多头（方案 B）的几何含义是：**把一份 d_{model} 维的「整空间注意力」切成 h 份「子空间注意力」，每份独立学一种关系，最后用 W_O 把它们融合回完整空间**。
QKV 的机制没有变化，只是在哪段子空间上做从「整段」变成「h 段」。

5.8 为什么这等价于「reshape + transpose」

工程实现里我们不会真的开 h 套小矩阵循环。利用一个数学等价性：

引理 5 (h 个小投影 = 1 个大投影 + 切块)**.** 把 h 套 $W_Q^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 沿列方向拼起来，得到一个大矩阵

$$W_Q \;=\; [W_Q^1, W_Q^2, \ldots, W_Q^h] \;\in\; \mathbb{R}^{d_{\text{model}} \times (h \cdot d_k)} = \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

则

$$XW_Q \;=\; [XW_Q^1,\; XW_Q^2,\; \ldots,\; XW_Q^h] \;\in\; \mathbb{R}^{n \times d_{\text{model}}},$$

也就是「一次大矩阵乘」的结果，**逐列切开正好是 h 段独立的 head_i 的 query**。

这一引理才是「reshape + transpose」的真正源头：

1. 把 h 套小投影写成一个大投影 W_Q，一次矩阵乘得到 $XW_Q \in \mathbb{R}^{n \times d_{\text{model}}}$；

2. 把最后一维重塑成 (h, d_k)，得到 $\mathbb{R}^{n\times h\times d_k}$；
3. 转置让 h 维移到前面，作为新的 batch 轴：$\mathbb{R}^{h\times n\times d_k}$。

到这一步，h 个头各自的 head_i 就可以用一次带 *batch* 的矩阵乘同时算完。

小结. 「reshape + transpose」不是什么炫技，它就是把数学上等价于「h 个独立小矩阵」的存储布局，摆成 *GPU* 喜欢的「带 *batch* 的张量」。意义和数学定义没有任何区别。

6 把整套推导收成一个清单

1. **目标**：构造 $f : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$，让每个输出位置都能依赖整段输入，且可微 / 可并行。
2. **第一次尝试**：$Y = \text{softmax}(XX^\top)X$。满足 4 条目标，但有三个缺陷：对称、key/value 绑死、无参数。
3. **修补三缺陷**：引入 $Q = XW_Q$, $K = XW_K$, $V = XW_V$。得到 $\text{Att}(Q, K, V) = \text{softmax}(QK^\top)V$。
4. **尺度修正**：维度变大会让点积方差变 d_k，softmax 饱和、梯度消失。除以 $\sqrt{d_k}$ 把方差稳定在 1，得到 $\text{Att}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k})V$。
5. **多头扩展**：一份注意力只能学一种关注模式。把 d_{model} 切成 h 份子空间，每份独立做 attention，最后拼接 + W_O。总参数量与单头几乎相同，只多了一个 W_O。
6. **工程等价**：h 套小投影 = 1 套大投影 + reshape + transpose，所以实现里只看到「一个大 Linear + reshape + 一次 batched matmul」。

本节交付

Attention 的整套公式都不是「定义」，而是「为了同时满足若干约束被推出来的结果」。你只要记住每一步的「为什么不能少」，就可以在白板上自己重新推一遍：

$$X \xrightarrow{W_Q, W_K, W_V} Q, K, V \xrightarrow{QK^\top/\sqrt{d_k}} \text{分数} \xrightarrow{\text{softmax}} A \xrightarrow{AV} \text{输出} \xrightarrow{W_O} Y.$$